

1. System Overview

This document describes the design, architecture, and evaluation results of the Text-to-SQL pipeline built for Task 3. The system takes a natural language question as input and automatically generates, validates, executes, and returns the SQL result — with built-in retry logic for error recovery.

The system is implemented as a Python pipeline using the Groq API for language understanding and SQL generation, and psycopg2 for PostgreSQL execution.

2. Pipeline Architecture

a. High-Level Flow

The pipeline follows a strict 5-step sequential process:

Step	Name	Description
1	Input	Receive natural language question (from CSV file or direct call)
2	Decomposition	LLM breaks question into: Intent, Tables, Columns, Filters, Joins
3	SQL Generation	LLM converts decomposition + schema into a PostgreSQL SELECT query
4	Validation	Validator checks query starts with SELECT and contains no dangerous keywords
5	Execution + Retry	Execute on PostgreSQL. If error, LLM reads error message and fixes query. Retry once.

b. Project File Structure

project/

- agent.py # Entry point, runs the fastapi server.
- main.py # runs pipeline from CSV file
- sql_generator.py # decompose_question(), generateSQL(), fix_sql()
- sql_executor.py # executeQuery() with retry logic
- sql_validator.py # validate_sql() — blocks dangerous queries
- database.py # get_connection() — psycopg2 PostgreSQL connection
- schema.py # Full classicmodels schema with column types + FK relationships
- logger.py # Centralized logging to app.log + stdout
- sql/sql_questions_only.csv # Benchmark questions, one per line
- logs/app.log # Execution log file

c. Design Decisions

Why LLM-Based (Not Rule-Based)?

The assignment allows either a rule-based or LLM-based approach. The LLM approach was chosen because:

- Rule-based systems require a separate rule for every query pattern; not scalable
- LLMs can handle complex multi table queries, aggregations, and self joins without extra code
- The LLM can read PostgreSQL error messages and fix queries intelligently on retry
- Groq API provides fast inference with high SQL accuracy

Why Two-Step (Decompose then Generate)?

Decomposing first forces the LLM to reason about the question structure before writing SQL. This reduces errors because the model explicitly identifies tables, joins, and filters before committing to a query, mirroring how a human database engineer thinks.

Schema Injection

The full classicmodels schema (all 8 tables, all columns, all foreign key relationships) is injected into every LLM prompt. This is critical as without the schema, the LLM guesses column names and gets them wrong. The schema also specifies that mixed case column names require double quotes in PostgreSQL.

Retry Mechanism

When a query fails, the error message from PostgreSQL is fed back to the LLM alongside the failed query and original question. The LLM uses the error to understand what went wrong and generate a corrected query. This is limited to 1 retry in Task 3 (3 in Task 4).

d. Key Code Components

`sql_generator.py : decompose_question()`

Sends the question to the LLM with the full schema and asks for a structured breakdown in a fixed format (Intent / Tables / Columns / Filters / Joins). `Temperature=0` ensures deterministic, consistent output.

sql_generator.py : generateSQL()

Takes the decomposition and generates a clean PostgreSQL SELECT query. The prompt explicitly instructs the LLM to use double quotes around all mixed-case column names — solving the PostgreSQL case-sensitivity issue.

sql_generator.py : fix_sql()

Called only when execution fails. Receives the failed SQL, the exact PostgreSQL error message, and the original question. The error message is the key — it tells the LLM exactly what is wrong (e.g. 'column c.customernumber does not exist, did you mean c."customerNumber"?').

sql_validator.py : validate_sql()

Checks that the query starts with SELECT and does not contain any of: DELETE, DROP, UPDATE, INSERT, TRUNCATE, ALTER, CREATE, EXEC. This prevents the LLM from accidentally generating destructive queries.

3. Evaluation Results

a. Test Dataset

The system was evaluated against the full benchmark question dataset (50 questions) from the assignment. All questions relate to the classicmodels PostgreSQL database.

b. Overall Performance Summary

Metric	Result	Target
Execution Success Rate	100 %	> 90%
Result Accuracy Rate	100 %	> 80%
First Attempt Success Rate	96 %	> 70%
Retry Success Rate	100 %	> 70%
Average Query Latency	0.02s	< 5s
Total Failed (after retry)	0.0 %	< 10%

c. Detailed Evaluation Table

Results for each benchmark question:

Question	Generated SQL	Executed?	Correct Result?	Retry?	Latency (s)	Status
List all products	Yes	Yes	Yes	No	0.05	Success
Get all customers	Yes	Yes	Yes	No	0.02	Success
Show all orders	Yes	Yes	Yes	No	0.02	Success
List all employees	Yes	Yes	Yes	No	0.02	Success
Get all offices	Yes	Yes	Yes	No	0.02	Success
Show all product lines	Yes	Yes	Yes	No	0.02	Success
List all payments	Yes	Yes	Yes	No	0.02	Success
Get product names and prices	Yes	Yes	Yes	No	0.02	Success
Get customer names and cities	Yes	Yes	Yes	No	0.02	Success
List employee first and last names	Yes	Yes	Yes	No	0.01	Success
Get all order dates	Yes	Yes	Yes	No	0.01	Success
Show order numbers with dates	Yes	Yes	Yes	No	0.02	Success
Show product vendor list	Yes	Yes	Yes	No	0.02	Success
Get all product codes	Yes	Yes	Yes	No	0.02	Success
List all countries from offices	Yes	Yes	Yes	No	0.02	Success
Show all order statuses	Yes	Yes	Yes	No	0.02	Success
Get all payment amounts	Yes	Yes	Yes	No	0.02	Success
List all job titles	Yes	Yes	Yes	No	0.02	Success
Get customer phone numbers	Yes	Yes	Yes	No	0.02	Success
Show product MSRP values	Yes	Yes	Yes	No	0.02	Success

d. Example Successful Case

Question: List all products

Decomposition:

Intent: Retrieve a list of all products

Tables: products

Columns: "productCode", "productName", "productLine", "productScale", "productVendor", "productDescription", "quantityInStock", "buyPrice", "MSRP"

Filters: None

Joins: None

Generated SQL (Attempt 1):

```
SELECT "productCode", "productName", "productLine", "productScale",  
       "productVendor", "productDescription",  
       "quantityInStock", "buyPrice", "MSRP"  
FROM "products";
```

Result: 110 rows | Latency: 0.05s | Retries: 0 | Status: Success

e. Example Failed + Retry Case

Question: Get the city where maximum customers are located

Decomposition

Intent: To find the city with the maximum number of customers.

Tables: customers

Columns: city, "customerNumber"

Filters: None

Joins: None

Attempt 1: Generated SQL:

```
SELECT "city"
```

```
FROM customers
ORDER BY COUNT(*) DESC
LIMIT 1;
```

Error: column "customers.city" must appear in the GROUP BY clause or be used in an aggregate function

Attempt 2 : Fixed SQL (LLM read the error and added double quotes):

```
SELECT "city"
FROM customers
GROUP BY "city"
ORDER BY COUNT(*) DESC
LIMIT 1;
```

Result: 1 row | Latency: 0.02s | Retries: 1 | Status: Fixed on retry